

Capture the Flag 3: Data-Driven UI

Goal: Separate application data from the UI and handle multiple pieces of data in a usable interface.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- **Explain** an error (not **fix** your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before modifying your application.

Create:

```
feature/data-ui
```

Screenshot:

- Terminal showing your current branch

🚩 Flag 1 — Identify the Data

Objective: Identify the appropriate Dart data type for the information in your profile.

Research basic Dart data types and identify at least **four different types** that could be used in your application.

Examples:

String
int
double
bool

Create a table identifying the information and its appropriate data type.

Create:

- Your identified data
- The corresponding data types

String	name
Int	age
Double	height
bool	status

Flag 2 — Create Your Data

Objective: Create appropriately typed variables for your profile.

Create variables for at least:

- Name
- Course & Section
- Age
- Hobby
- Height
- Student status

Your data should exist **separately from your UI widgets**. (Declare variables separately from your UI, not directly inside your widgets)

Screenshot:

- Your variables
- Their data types
- Application still running

4:10

48.9 KB/s

My First Flutter Application



Leachim Dela Cerna

BSIT-3

Age: 21

Hobby: Dancing

Height: 170.0 cm

Student: Yes

```
String name = 'Leachim Dela Cerna';  
String courseSection = 'BSIT-3';  
int age = 21;  
String hobby = 'Dancing';  
double height = 170.0;  
bool isStudent = true;
```

Flag 3 — Connect Data to UI

Objective: Replace your hardcoded information with your variables.

Your profile should now display information using the variables you created.

Test:

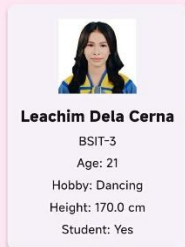
Change the value of one of your variables.

The UI should automatically display the new value.

Screenshot:

- Variables
- Widgets displaying the variables
- Updated application

4:10 4G LTE 50%
My First Flutter Application



```
String name = 'Leachim Dela Cerna';  
String courseSection = 'BSIT-3';  
int age = 21;  
String hobby = 'Dancing'; // Changed v  
double height = 170.0;  
bool isStudent = true;
```

```
Widget build(BuildContext context) {  
  return MaterialApp(  
    debugShowCheckedModeBanner: false,  
    title: 'My First Flutter Application',  
    home: Scaffold(  
      backgroundColor: const Color(0xFFFFE6F0),  
  
      appBar: AppBar(  
        title: const Text(  
          'My First Flutter Application',  
          style: TextStyle(color: Colors.black, fontSize: 16),  
        ), // Text  
        backgroundColor: Colors.white,  
      ), // AppBar  
  
      body: Center(  
        child: Card(  
          margin: const EdgeInsets.all(20),  
          elevation: 4,  
          child: Padding(  
            padding: const EdgeInsets.all(20),  
            child: Column(  
              mainAxisAlignment: MainAxisAlignment.min,  
              crossAxisAlignment: CrossAxisAlignment.center,  
              children: [  
                // Profile picture  
                Image.asset('assets/DelaCerna.jpg', width: 120, height: 120),  
  
                const SizedBox(height: 15),  
  
                // Name - using variable  
                Text(  
                  name,  
                  style: const TextStyle(  
                    fontSize: 22,  
                    fontWeight: FontWeight.bold,  
                  ), // TextStyle  
                ), // Text  
  
                const SizedBox(height: 8),  
  
                // Course & Section - using variable  
                Text(courseSection, style: const TextStyle(fontSize: 16)),  
  
                const SizedBox(height: 8),  
  
                // Age - using variable  
                Text('Age: $age', style: const TextStyle(fontSize: 16)),  
  
                const SizedBox(height: 8),  
  
                // Hobby - using variable  
                Text('Hobby: $hobby', style: const TextStyle(fontSize: 16)),  
  
                const SizedBox(height: 8),  
  
                // Height - using variable  
                Text(  
                  'Height: ${height.toStringAsFixed(1)} cm',  
                  style: const TextStyle(fontSize: 16),  
                ), // Text  
  
                const SizedBox(height: 8),  
  
                // Student Status - using variable  
                Text(  
                  'Student: ${isStudent ? "Yes" : "No"}',  
                  style: const TextStyle(fontSize: 16),  
                ),  
              ],  
            ),  
          ),  
        ),  
      ),  
    ),  
  );  
}
```

Flag 4 — Make the Image Data-Driven

Objective: Make your profile image part of your data.

Create a variable containing the path to your profile image.

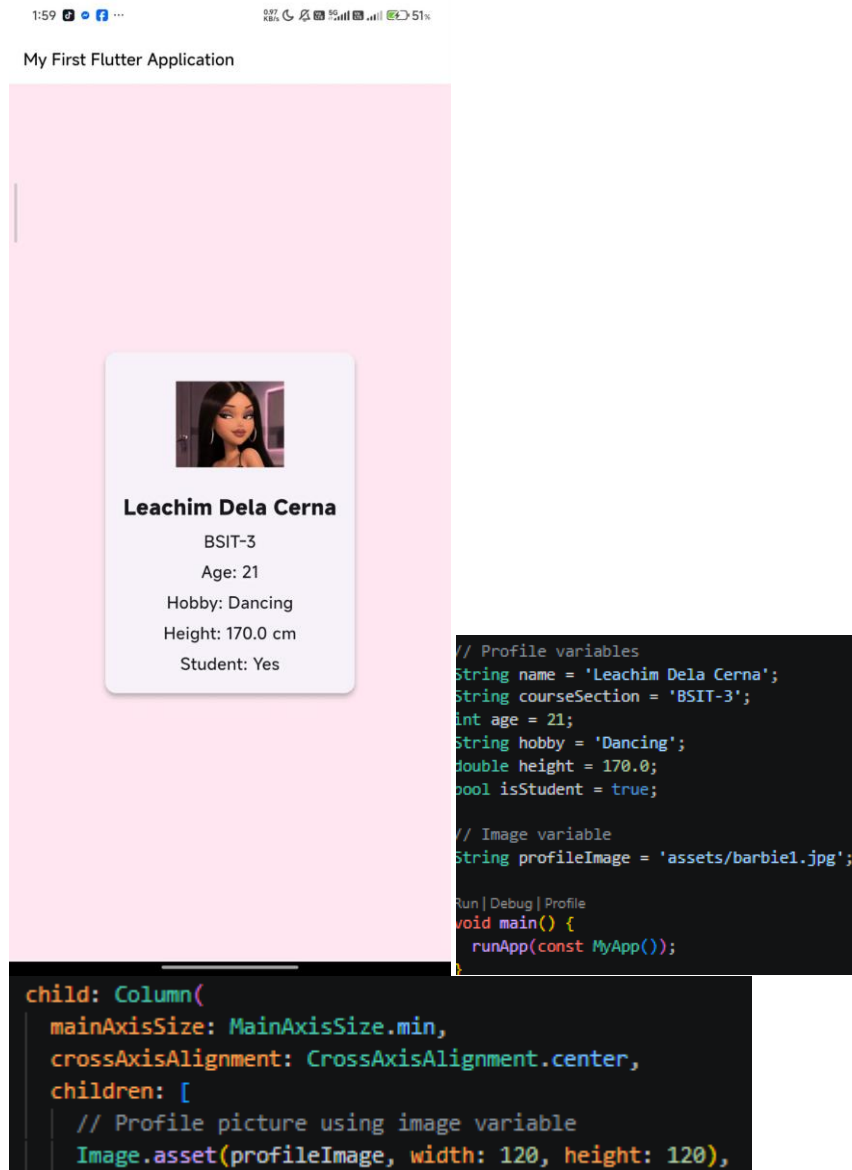
Use the variable when displaying the image.

Then change the variable to another registered image.

The image displayed by the application should change.

Screenshot:

- Image variable
- Image widget using the variable
- Updated application



🚩 Flag 5 — Five Profiles, One Design

Objective: Create at least **five different profile cards** using the same visual design.

Each profile must contain at least:

- Image
- Name
- Course & Section
- Age
- Hobby

The five profiles must contain different information.

Important

All five profiles should use the **same visual design**.

You are demonstrating that the same UI can represent different data.

Screenshot:

- Your 5 different profile data

```
class Profile {
    String image;
    String name;
    String courseSection;
    int age;
    String hobby;

    Profile({
        required this.image,
        required this.name,
        required this.courseSection,
        required this.age,
        required this.hobby,
    });
}

List<Profile> profiles = [
    Profile(
        image: 'assets/DelaCerna.jpg',
        name: 'Leachim Dela Cerna',
        courseSection: 'BSIT-3',
        age: 21,
        hobby: 'Dancing',
    ),
    Profile(
        image: 'assets/barbie.jpg',
        name: 'Julia Kong',
        courseSection: 'BSIT-3',
        age: 20,
        hobby: 'Singing',
    ),
    Profile(
        image: 'assets/barbiel.jpg',
        name: 'Mav Tajanlangit',
        courseSection: 'BSIT-3',
        age: 21,
        hobby: 'Drawing',
    ),
    Profile(
        image: 'assets/barbie3.jpg',
        name: 'Dom Ocarol',
        courseSection: 'BSIT-3',
        age: 20,
        hobby: 'Watching Movies',
    ),
    Profile(
        image: 'assets/barbie4.jpg',
        name: 'Tien Verano',
        courseSection: 'BSIT-3',
        age: 22,
        hobby: 'Gaming',
    ),
];
```

🚩 Flag 6 — THE HIDDEN PROBLEM

You didn't see your UI?

Objective: Make all five profiles accessible to the user.

You just added five profile cards.

Now run your application.

Something is wrong.

Look carefully at your emulator.

Can you see all five profiles?

Maybe you can see the first one...

Maybe part of the second...

But where are the rest?

That's right.

You can't see them.

Your UI is larger than the available screen.

The profiles exist.

The data exists.

The widgets exist.

But the user can't access everything.

Your Mission:

Figure out how Flutter allows users to **scroll through content that is larger than the available screen**.

Research Flutter's **scrolling widgets** and determine which one is appropriate for your current layout.

You must make it possible to scroll through all five profiles.

Requirement:

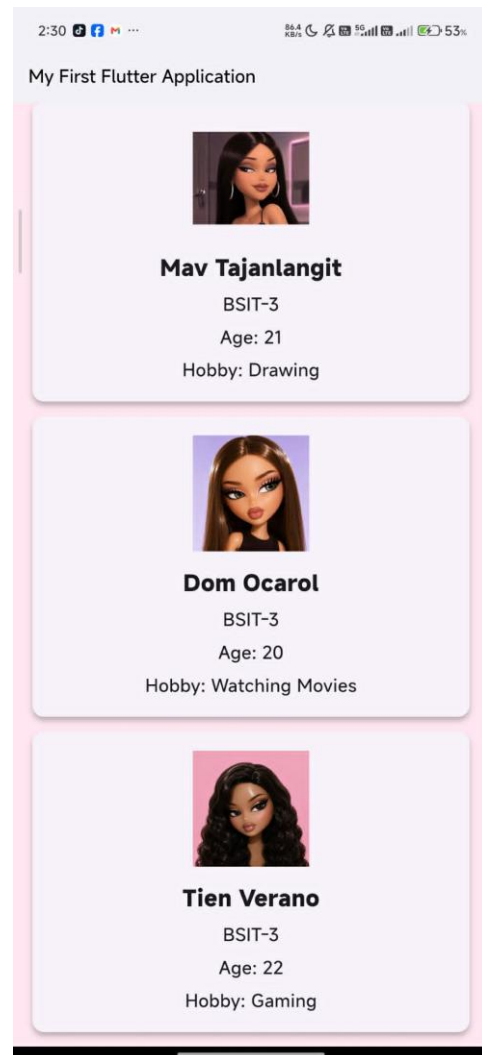
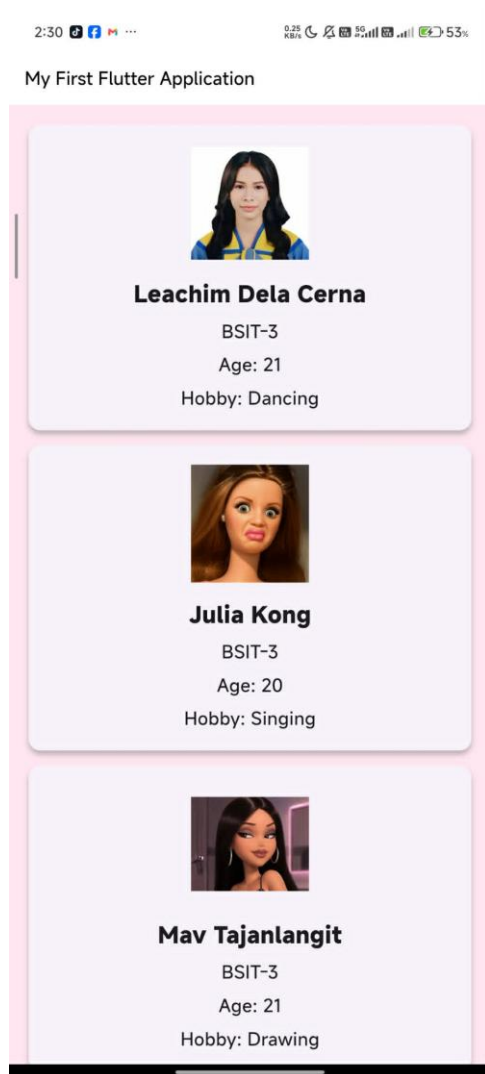
The user must be able to:

Scroll → see Profile 1 → Profile 2 → Profile 3 → Profile 4 → Profile 5

without changing the size of the screen.

Screenshot:

- The scrolling widget in your code
- Top portion of the application
- The rest of the portion of the application showing the later profiles



Flag 7 — Survive the Database

Objective: Prepare your UI for incomplete data.

Now imagine your profiles aren't manually created.

Someday, the data will come from a database.

And real data isn't always perfect.

Some information may be:

- Missing
- Empty
- `null`
- Not provided

Create at least **three profiles with missing information**.

For example:

Profile A

Name: Maria

Course: BSIT

Hobby: missing

Profile B

Name: Juan

Course: missing

Hobby: Gaming

Profile C

Name: missing

Course: MMA

Hobby: Drawing

Your application must **not crash** when information is missing.

Instead, provide an appropriate fallback.

Examples:

Hobby: Not provided

Course: Unknown

Challenge

Your placeholders should look intentional and clean.

Screenshot:

- Incomplete profile data
- Application displaying fallback values
- Application still functioning correctly

```

        name: 'Leachim Dela Cerna',
        courseSection: 'BSIT-3',
        age: 21,
        hobby: 'Dancing',
    ),

    // Complete Profile
    Profile(
        image: 'assets/barbie.jpg',
        name: 'Julia Kong',
        courseSection: 'BSIT-3',
        age: 20,
        hobby: 'Singing',
    ),

    // Profile with missing Hobby
    Profile(
        image: 'assets/barbie1.jpg',
        name: 'Mav Tajanlangit',
        courseSection: 'BSIT-3',
        age: 21,
        hobby: null,
    ),

    // Profile with missing Course
    Profile(
        image: 'assets/barbie3.jpg',
        name: 'Dom Ocarol',
        courseSection: null,
        age: 20,
        hobby: 'Watching Movies',
    ),

    // Profile with missing Name
    Profile(
        image: 'assets/barbie4.jpg',
        name: null,
        courseSection: 'BSIT-3',
        age: 22,
        hobby: 'Gaming',
    ),
};

```

```

class Profile {
    String image;
    String? name;
    String? courseSection;
    int? age;
    String? hobby;

    Profile({
        required this.image,
        this.name,
        this.courseSection,
        this.age,
        this.hobby,
    });
}

```

```

String showData(String? value) {
    if (value == null || value.trim().isEmpty) {
        return 'Not provided';
    }

    return value;
}

```

🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Data-Driven UI CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/data-ui`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

```
PS C:\Users\Student\Desktop\IT314AB_DelaCerna\my_first_flutter_app> git branch
* feature/data-ui
  feature/widgets
  main
PS C:\Users\Student\Desktop\IT314AB_DelaCerna\my_first_flutter_app> git status
On branch feature/data-ui
Your branch is up to date with 'origin/feature/data-ui'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   lib/main.dart
    modified:   pubspec.yaml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    assets/barbie.jpg
    assets/barbie1.jpg
    assets/barbie3.jpg
    assets/barbie4.jpg

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Student\Desktop\IT314AB_DelaCerna\my_first_flutter_app> git add .
PS C:\Users\Student\Desktop\IT314AB_DelaCerna\my_first_flutter_app> git commit -m "feat: submit Data-Driven UI"
[feature/data-ui 051ea57] feat: submit Data-Driven UI
6 files changed, 151 insertions(+), 233 deletions(-)
create mode 100644 my_first_flutter_app/assets/barbie.jpg
create mode 100644 my_first_flutter_app/assets/barbie1.jpg
create mode 100644 my_first_flutter_app/assets/barbie3.jpg
create mode 100644 my_first_flutter_app/assets/barbie4.jpg
PS C:\Users\Student\Desktop\IT314AB_DelaCerna\my_first_flutter_app> git push
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 12 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (11/11), 180.54 KiB | 25.79 MiB/s, done.
Total 11 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/chm0-boop/IT314AB_DelaCerna.git
   28c7376..051ea57  feature/data-ui -> feature/data-ui
PS C:\Users\Student\Desktop\IT314AB_DelaCerna\my_first_flutter_app> |
```